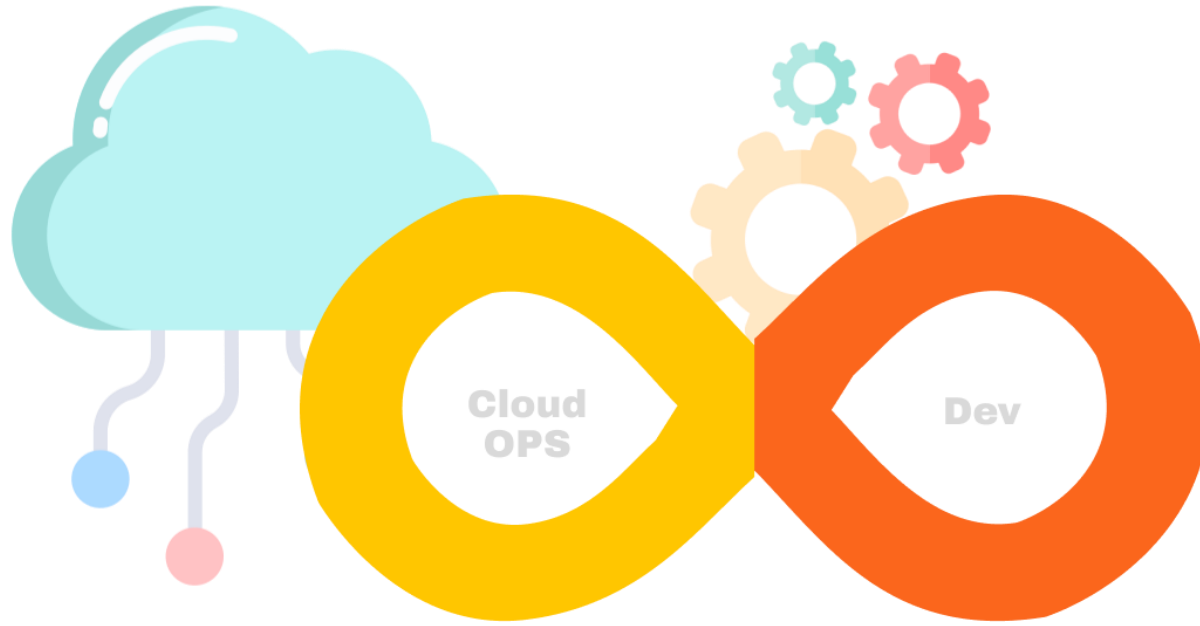




Containerization

Muhammad Ali

Course : DevOps and Cloud Computing Foundations
Department of Computer Science, SCIT, Beaconhouse National University
(BNU)

Docker Networking

A Complete Professional Guide

Understanding Network Types, Architecture & Best Practices



D O C K E R



Agenda

01

What is Docker Networking?

Fundamentals and core concepts

02

Types of Docker Networks

Bridge, Host, None, Overlay, Macvlan, IPvlan

03

Network Architecture Diagrams

Visual deep-dive into each network type

04

When to Use Which Network

Decision framework and best practices

01

What is Docker Networking?

Understanding the fundamentals of container communication



What is Docker Networking?

Docker networking is the communication layer that enables containers to interact with each other, the host system, and external networks. It provides isolated and configurable network environments for containerized applications.

Isolation

Each container gets its own network namespace with private IP

Connectivity

Containers communicate using virtual bridges and overlays

Flexibility

Multiple driver types for different use cases and architectures



How Docker Networking Works



Network Namespace

Each container gets an isolated network stack with its own interfaces, routing tables, and firewall rules.



Virtual Ethernet (veth)

A pair of virtual interfaces connecting the container's namespace to the host bridge.



Network Drivers

Pluggable backends (bridge, host, overlay, etc.) that implement different networking models.



DNS Resolution

Docker's embedded DNS server (127.0.0.11) enables service discovery by container name.



Essential Docker Network Commands

Command	Description
<code>docker network ls</code>	List all networks
<code>docker network create <name></code>	Create a new network
<code>docker network inspect <name></code>	View network details
<code>docker network connect <net> <ctr></code>	Connect container to network
<code>docker network disconnect <net> <ctr></code>	Disconnect container
<code>docker network rm <name></code>	Remove a network
<code>docker network prune</code>	Remove unused networks

02

Types of Docker Networks

Six built-in network drivers for every use case



Docker Network Types Overview

Bridge

Default. Isolated network on single host.

Host

Shares host network stack directly.

None

Complete network isolation.

Overlay

Multi-host communication (Swarm).

Macvlan

Unique MAC per container, L2 access.

IPvlan

Shared MAC, unique IPs, L2/L3.

Bridge Network

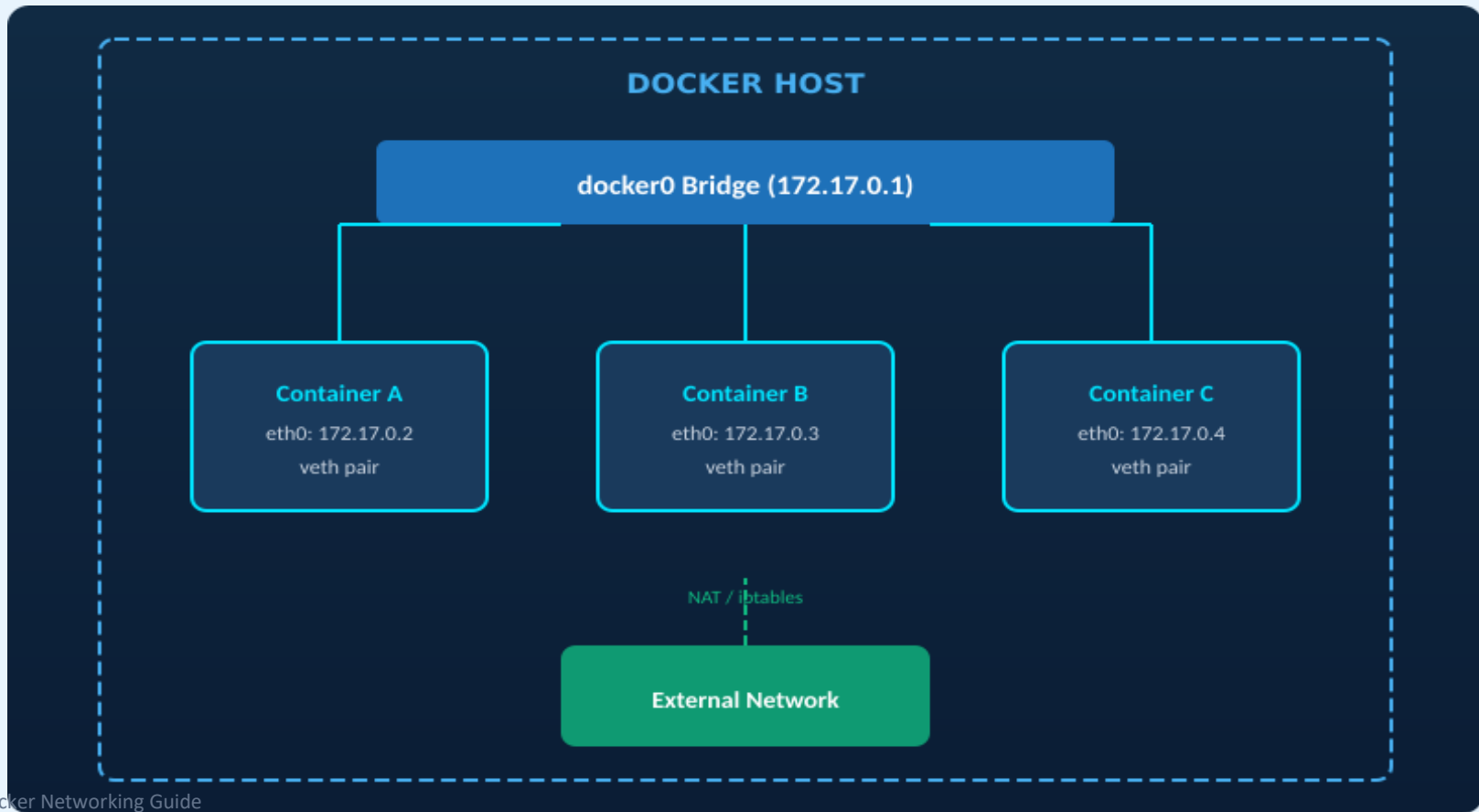
DEFAULT

The bridge network is Docker's default network driver. It creates an isolated network on a single host using a Linux bridge (docker0). Containers on the same bridge can communicate via IP addresses, and user-defined bridges also support DNS-based discovery.

- ✓ Automatic IP allocation from subnet
- ✓ Containers communicate via internal IPs
- ✓ Port mapping required for external access (-p flag)
- ✓ User-defined bridges support DNS service discovery
- ✓ Network isolation between different bridges
- ✓ NAT-based outbound connectivity via host

```
docker network create --driver bridge my_bridge
```

Bridge Network Architecture



Host Network

PERFORMANCE

The host network mode removes network isolation between the container and the Docker host. The container shares the host's network namespace directly, meaning it uses the host's IP address and port space with zero NAT overhead.



No network isolation — container uses host IP



Best performance — zero NAT overhead



Port conflicts possible with host services



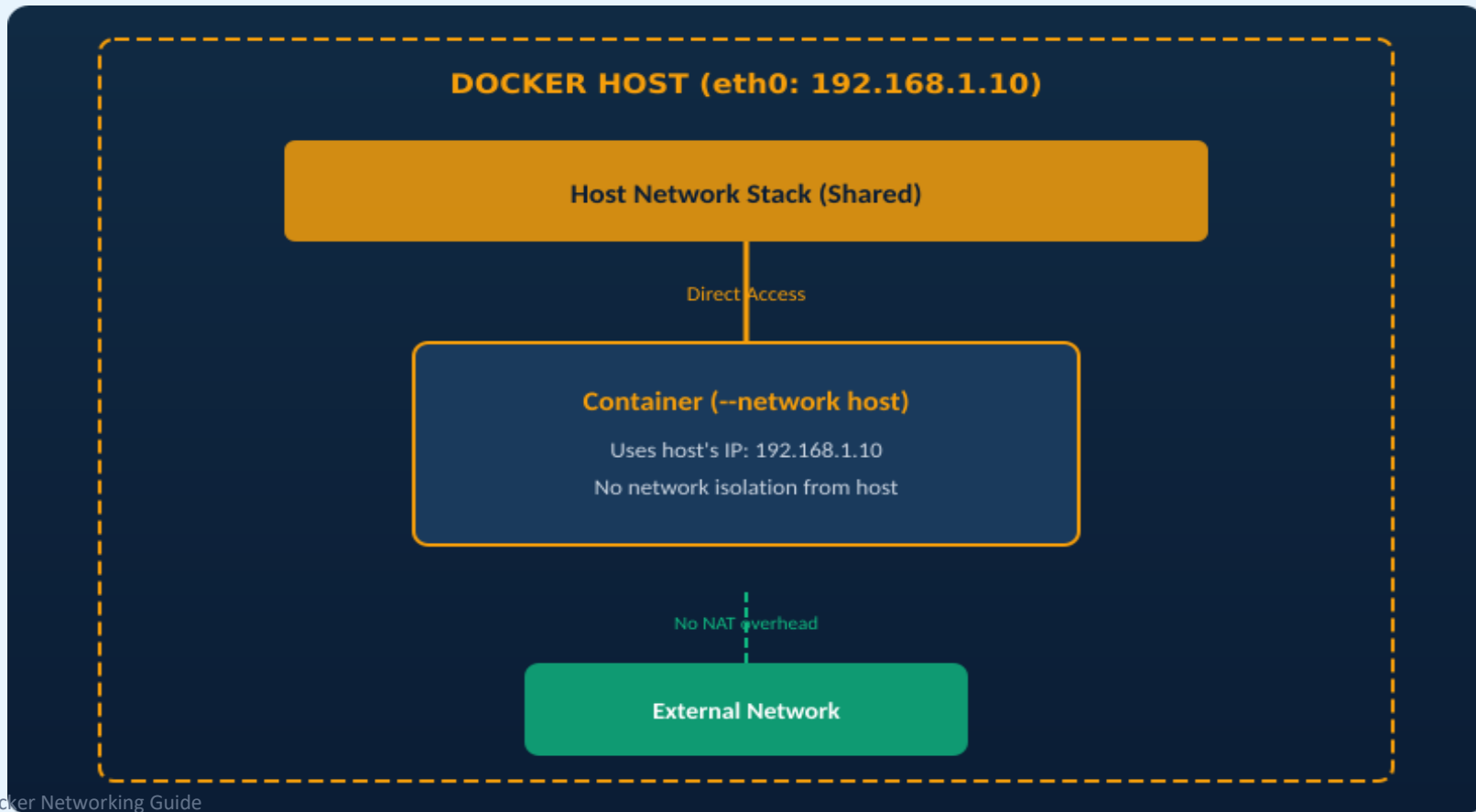
Ideal for high-throughput / low-latency workloads



Not available on Docker Desktop (Mac/Windows)

```
docker run --network host nginx
```

Host Network Architecture



None Network

ISOLATED

The none network completely disables networking for a container. The container only has a loopback interface (127.0.0.1) and cannot communicate with any other container, the host, or external networks.



Only loopback (lo) interface available



Maximum security — no network attack surface



Ideal for batch processing & offline computation



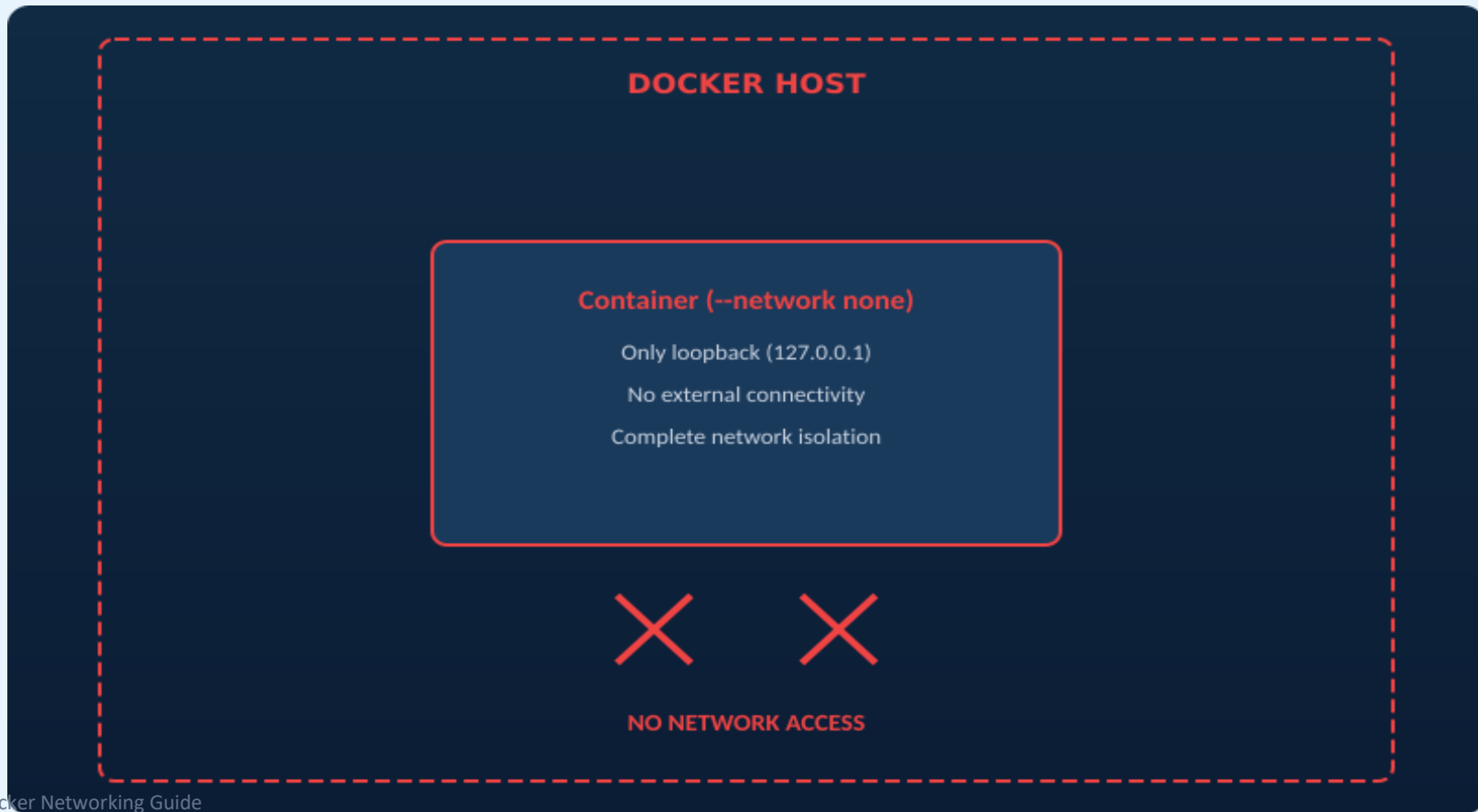
Used for sensitive data processing in isolation



Container cannot be accessed remotely

```
docker run --network none alpine
```

None Network Architecture



Overlay Network

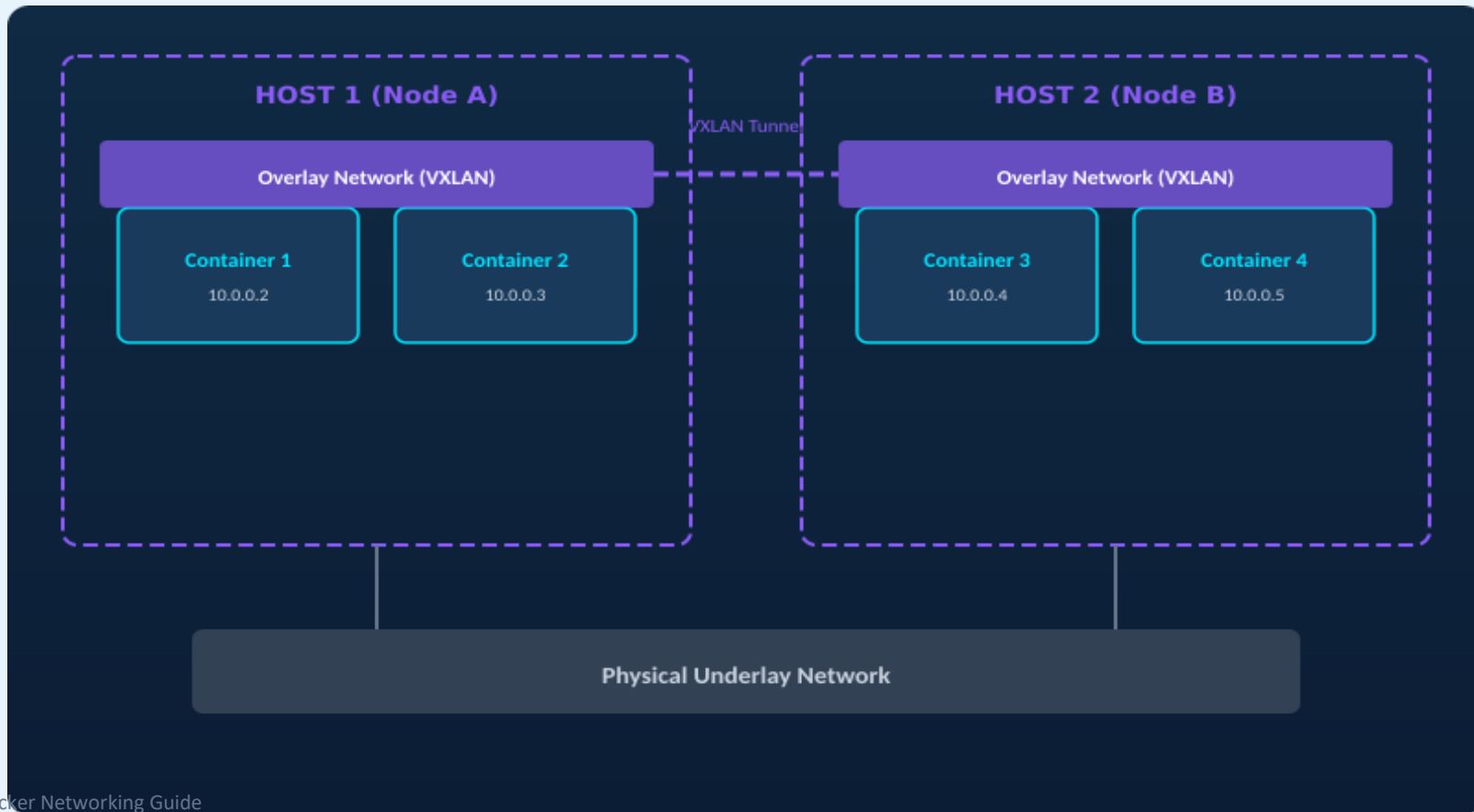
MULTI-HOST

Overlay networks enable containers running on different Docker hosts to communicate as if they were on the same local network. Uses VXLAN tunneling to encapsulate packets across the physical underlay.

- ✓ Spans multiple Docker hosts (Swarm / Kubernetes)
- ✓ Uses VXLAN encapsulation for L2 over L3
- ✓ Automatic service discovery and load balancing
- ✓ Encrypted traffic option (`--opt encrypted`)
- ✓ Essential for microservices at scale
- ✓ Requires Swarm mode or external KV store

```
docker network create --driver overlay my_overlay
```


Overlay Network Architecture



Macvlan Network

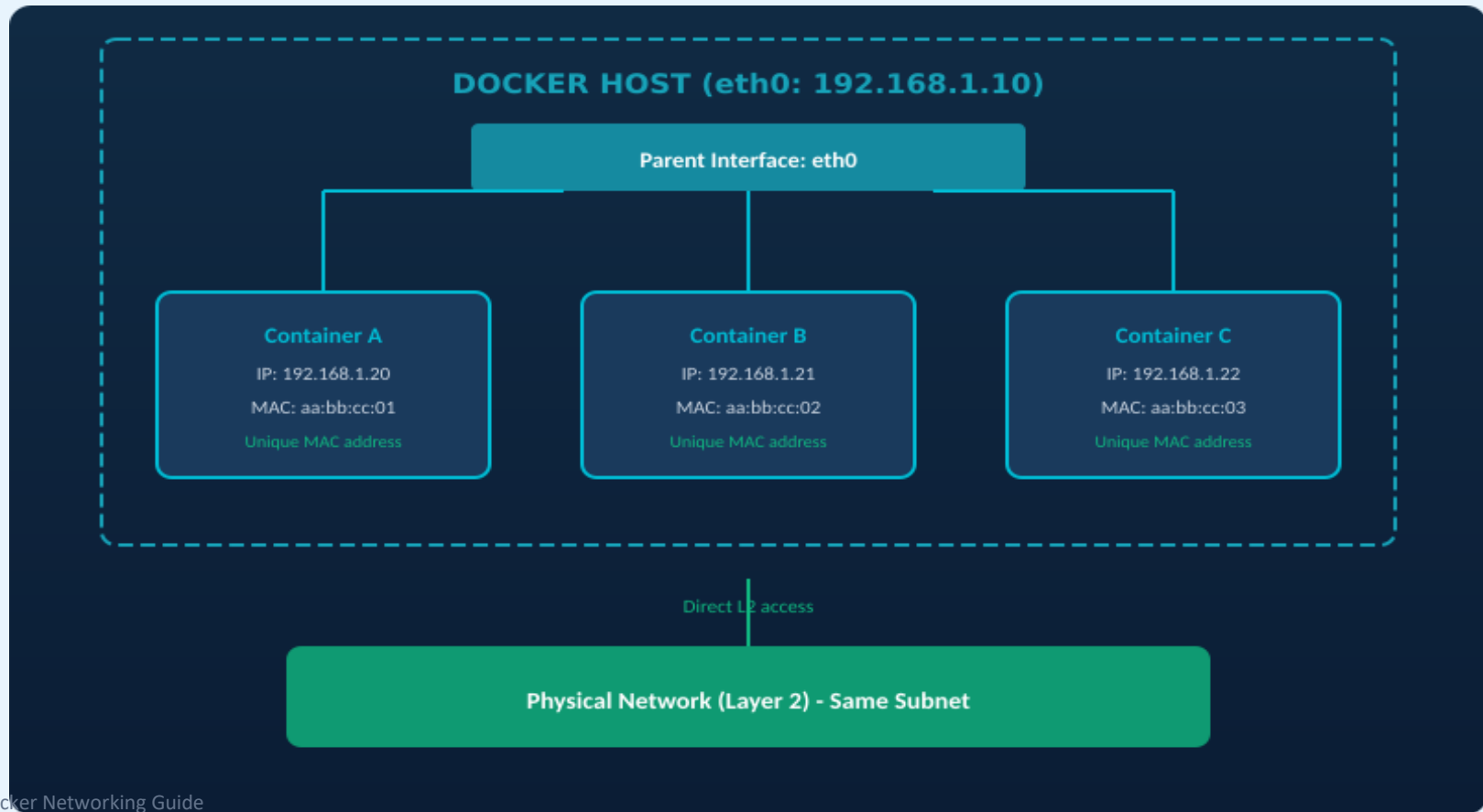
LAYER 2

Macvlan assigns a unique MAC address to each container, making it appear as a physical device on the network. Containers get direct Layer 2 access to the physical network without NAT or bridge overhead.

- ✓ Each container gets a unique MAC address
- ✓ Direct Layer 2 connectivity to physical network
- ✓ No NAT — containers appear as physical hosts
- ✓ Requires promiscuous mode on host NIC
- ✓ Ideal for legacy apps expecting physical IPs
- ✓ Container-to-host communication requires extra config

```
docker network create -d macvlan --subnet=192.168.1.0/24 -o parent=eth0 my_mac
```

Macvlan Network Architecture



IPvlan Network

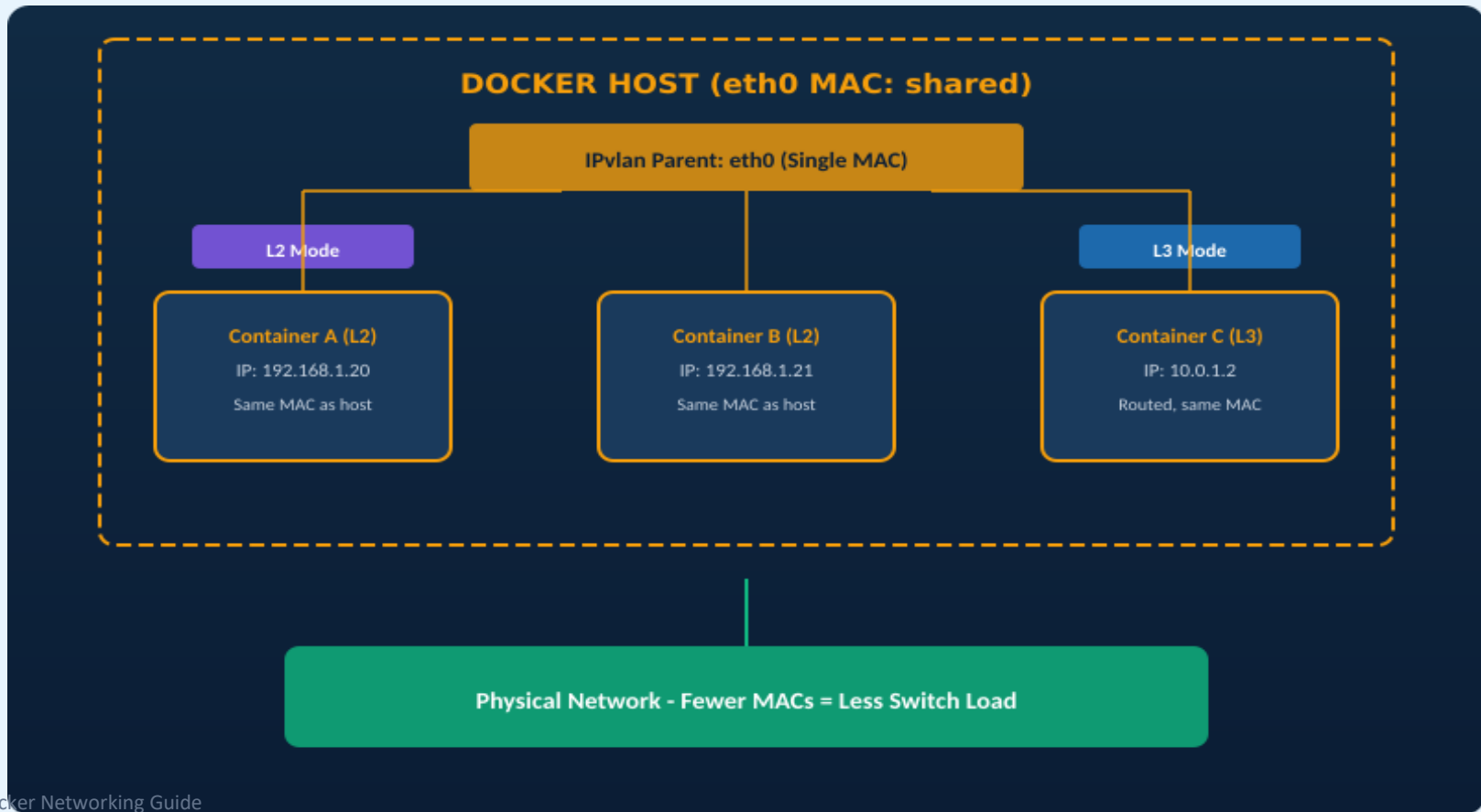
L2 / L3

IPvlan is similar to Macvlan but all containers share the host's MAC address while having unique IPs. This reduces MAC address table pressure on network switches, making it ideal for environments with MAC restrictions.

- ✓ All containers share the host's MAC address
- ✓ L2 mode: same subnet, bridged connectivity
- ✓ L3 mode: different subnets, routed connectivity
- ✓ Lower switch MAC table overhead than Macvlan
- ✓ Does not require promiscuous mode
- ✓ Best for environments with MAC address restrictions

```
docker network create -d ipvlan --subnet=192.168.1.0/24 -o parent=eth0 my_ipvlan
```

IPvlan Network Architecture



04

When to Use Which Network?

Choosing the right driver for your use case

Network Driver Comparison

Feature	Bridge	Host	None	Overlay	Macvlan
Isolation	High	None	Maximum	High	Medium
Performance	Good	Best	N/A	Good	Excellent
Multi-Host	No	No	No	Yes	No
DNS Discovery	Yes*	No	No	Yes	No
Port Mapping	Required	N/A	N/A	Optional	No
External Access	Via NAT	Direct	None	Via NAT	Direct L2
Complexity	Low	Low	Low	High	Medium

** User-defined bridges only*



Decision Guide: Which Network to Use?

Single host, standard web apps, development

Bridge

Maximum throughput, low latency, monitoring tools

Host

Security-critical batch jobs, offline processing

None

Multi-host clusters, Swarm/K8s microservices

Overlay

Legacy apps needing real IPs, direct L2 access

Macvlan

Environments with MAC limits, cloud restrictions

IPvlan

Real-World Use Case Scenarios



E-Commerce Platform

Frontend, API, and DB on bridge network with internal DNS. Overlay for scaling across nodes.



IoT Gateway

Macvlan to appear as physical device on industrial network. Legacy SCADA compatibility.



CI/CD Pipeline

Bridge for build containers. None network for security scanning stages. Host for performance tests.



Data Processing

None network for sensitive batch jobs. Overlay for distributed Spark/Hadoop clusters.



Docker Networking Best Practices



Always use user-defined bridge networks instead of the default bridge



Leverage built-in DNS — refer to containers by name, not IP



Use overlay networks with encryption for sensitive inter-host traffic



Apply the principle of least privilege — only expose necessary ports



Use docker network prune regularly to clean up unused networks



Implement network policies to control container-to-container traffic



Monitor network performance and latency between containers



Document your network architecture for team onboarding



Common Pitfalls to Avoid



Using default bridge for production

No DNS resolution, limited isolation. Always create user-defined networks.



Exposing unnecessary ports

Reduces security. Only publish ports that need external access.



Ignoring network driver limits

Host mode has port conflicts. Overlay needs Swarm. Plan ahead.



Hardcoding container IPs

IPs change on restart. Use DNS names or service discovery.



Skipping network monitoring

Latency issues go unnoticed. Use tools like cAdvisor or Prometheus.

Demo Commands

- `docker run -d --name sakila-app --network sakilanetwork -p 5000:5000 sakila-latest`
- `docker run -d --name mysql-container --network sakilanetwork -e MYSQL_ROOT_PASSWORD=admin -e MYSQL_DATABASE=sakila`
- `mysqlPath of dump.sql file`
- `docker exec -i mysql-container mysql -u root -padmin sakila < sakila.sql`

Key Takeaways

- 1 Bridge is your default go-to for single-host container communication
- 2 Host mode trades isolation for raw performance — use judiciously
- 3 None provides maximum security for workloads that need no networking
- 4 Overlay is essential for multi-host architectures (Swarm, K8s)
- 5 Macvlan/IPvlan let containers appear as physical hosts on the network
- 6 Always prefer user-defined networks over Docker defaults



Thank You!

Docker Networking - Complete Professional Guide

Questions & Discussion